How Computers Work · Section 3

Representing Text: ASCII, Unicode, and Base64

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. How Text Becomes Numbers 1

2. ASCII 2

3. The Problem of Other Languages 2

4. Unicode and UTF-8 3

5. Khmer and Other Scripts in Unicode 3

6. Base64: Encoding, Not Encryption 4

7. URL and Percent Encoding 5

8. Why Encoding Matters in Security 5

Common Mistakes 6

Summary 6

Check Your Understanding 7

Practice 7

What You Will Learn

- How text becomes numbers.
- ASCII: the original text encoding.
- Why ASCII was not enough, and how Unicode fixes it.
- UTF-8, and how Khmer and other scripts are stored.
- Base64: encoding that is often mistaken for encryption.
- URL (percent) encoding.
- Why encoding matters in security.

Before You Start

Read lessons 1 and 2. You know everything is bytes, and you can read a hex dump. This lesson answers the question lesson 2 left open: why is the letter `H` the byte `0x48`? The console examples use Python, `base64`, and `xxd`, shown exactly as they ran.

A key idea for the whole lesson: **encoding is a rule for turning information into bytes, and back.** It is not a secret and not protection; it is an agreement about how to read bytes. Confusing encoding with encryption is one of the most common beginner mistakes in security, and we clear it up here.

---

1. How Text Becomes Numbers

A computer only stores numbers (lesson 1), so to store the letter `H`, it must store a *number* that stands for `H`. For this to work, everyone has to agree on which number means which character. That agreement is a **character encoding**: a table that maps each character to a number (and the bytes that hold it).

Think of it like a codebook both sides share: the writer turns `H` into a number using the table, and the reader turns that number back into `H` using the same table. If the two use *different* tables, the reader gets the wrong characters, which is exactly what "garbled text" is.

So the question "why is `H` the byte `0x48`?" has a simple answer: **because a standard table says so**, and everyone agreed to use it. That table, for the basic English characters, is ASCII.

2. ASCII

ASCII (American Standard Code for Information Interchange) is the original character encoding, from the 1960s. It assigns a number from 0 to 127 to each basic English character: the letters A–Z and a–z, the digits 0–9, punctuation, the space, and some invisible "control" characters.

Because the biggest ASCII value is 127, every ASCII character fits in **7 bits**, and so comfortably in one byte. Some landmark values, read for real:

```
$ python3 -c 'print(ord("H"), hex(ord("H"))); print(ord("A"), ord("a"), ord("0"),
ord(" "))'
72 0x48
65 97 48 32
```

So `H` is 72, which is `0x48`, exactly the byte we saw in the lesson-2 hex dump. A capital `A` is 65, a lowercase `a` is 97, the digit character `0` is 48, and a space is 32.

Two patterns worth noticing, because they come up in real work:

- **The digit character `0` is the number 48, not 0.** The *character* `'0'` and the *number* `0` are different things. This catches people constantly.
- **Uppercase and lowercase differ by 32.** `A` is 65 and `a` is 97; `65 + 32 = 97`. That is why changing case is a simple, fast operation.

ASCII is small, simple, and still the foundation: the first 128 values of every modern encoding are still ASCII. But 128 characters cannot hold the world's writing.

3. The Problem of Other Languages

ASCII was designed for American English. It has no `é`, no `ñ`, no Chinese characters, no Arabic, and crucially for us, **no Khmer**. For decades, each language group invented its own incompatible tables that reused the numbers above 127 for different characters. The result was chaos: text written with one table turned into garbage when read with another, because the same byte meant a different character in each.

This is more than an inconvenience. When the same bytes can be read as different characters depending on the assumed table, you get **ambiguity**, and attackers exploit ambiguity. Different parts of a system interpreting the same bytes differently is the seed of real security bugs (for example, sneaking a dangerous character past a filter that reads the bytes one way while the target reads them another).

The world needed **one** table big enough for every character ever used. That is Unicode.

4. Unicode and UTF-8

Unicode is a single, giant table that aims to give every character in every writing system its own unique number, called a **code point**. There is room for over a million code points; the code point for `H` is still 72 (Unicode kept ASCII at the bottom), while Khmer, emoji, and Chinese characters all have their own code points far above 127.

But a code point is just a number. The computer still has to store it as *bytes*, and here is the clever part. **UTF-8** is the rule, by far the most common, for turning Unicode code points into bytes. Its design is brilliant for two reasons:

- **It is backward-compatible with ASCII.** Any ASCII character (code point 0–127) is stored as the *same single byte* as before. So all existing English text is already valid UTF-8, unchanged.
- **It uses more bytes only when needed.** Characters beyond ASCII use two, three, or four bytes.

See it directly. A plain `A` is one byte; `é` takes two; the Khmer letter `ñ` takes three:

```
$ python3 -c 'print("A".encode("utf-8").hex(" ")); print("é".encode("utf-8").hex("
")); print("ñ".encode("utf-8").hex(" "))'
41
c3 a9
e1 9e 80
```

This has a practical consequence you must internalize: **one character is not always one byte.** In a hex dump, `é` shows up as two bytes, which do not individually look like text:

```
$ printf 'é' | xxd
00000000: c3a9                                     ..
```

The `..` on the right means "these bytes are not printable on their own". So when you count characters, measure text length, or chop a string, bytes and characters can differ. This matters for Khmer, where each letter is several bytes, and it is a frequent source of bugs and of security issues in text handling. **UTF-8 everywhere** is the modern default, and the right choice.

5. Khmer and Other Scripts in Unicode

Khmer has its own block of code points in Unicode, so Cambodian text is fully supported, and that is why these very lessons can be written in Khmer. A couple of points that matter when you handle Khmer text in security work:

- **Each Khmer character is typically three bytes in UTF-8** (as `ñ` showed above: `e1 9e 80`). A Khmer word that looks short on screen is many bytes long underneath.

- **What looks like one character can be several code points.** Khmer uses combining marks (for vowels and signs) that attach to a base consonant, so one visible cluster on screen may be built from several code points, and therefore many bytes. The same is true of accented letters and emoji in other scripts.

The takeaway is the same as before: **on-screen appearance, code points, and bytes are three different things.** A forensic analyst searching a disk for a Khmer name, or a developer validating a form, must know that "one character" is not a reliable unit. Always think in the layer you actually need: visible text, code points, or bytes.

6. Base64: Encoding, Not Encryption

Now a different kind of encoding, and the most important security lesson in this chapter. **Base64** is a way to represent *any* binary data (not just text) using only 64 safe, printable characters: A–Z, a–z, 0–9, and two symbols. It exists because some systems (old email, parts of the web, URLs) can only safely carry plain text, so binary data, images, keys, attachments, is Base64-encoded into text to pass through them, then decoded back on the other side.

Here it is, encoding then decoding the same text:

```
$ printf 'Mina Coffee' | base64
TWluYSBDb2ZmZWU=
$ printf 'TWluYSBDb2ZmZWU=' | base64 -d
Mina Coffee
```

You will recognize Base64 on sight after a while: a run of letters and digits, often ending in one or two `=` signs. Now the crucial point:

> **Base64 is encoding, not encryption. It provides no secrecy at all.**

Anyone can decode Base64 instantly, with no key, as the second command shows. It does not hide, protect, or secure anything; it only repackages data into safe characters. Beginners constantly see Base64 in a token or a config file and think the data is "encrypted" or safe. It is not. If you can read the Base64, you (and any attacker) can read the original in one step.

This distinction is central to security:

- **Encoding** (ASCII, UTF-8, Base64, URL-encoding) is a *public rule* for representing data. No secret, no protection. Reversible by anyone.
- **Encryption** (the Cryptography course) uses a *secret key* to make data unreadable without that key.

Whenever you see encoded data, read it; never assume it is protected. And never "secure" a secret by encoding it.

7. URL and Percent Encoding

One more everyday encoding, because you will meet it constantly in web security. Web addresses (URLs) can only use a limited set of characters. Characters outside that set, spaces, and symbols with special meaning (`&`, `=`, `/`, `?`), are replaced by a `%` followed by the character's byte in hex. This is **URL encoding** or **percent encoding**.

```
$ python3 -c 'import urllib.parse; print(urllib.parse.quote("a b&c=/?"))'
a%20b%26c%3D/%3F
```

Read it: the space became `%20` (because a space is byte `0x20`, which is 32, exactly the ASCII space from topic 2), `&` became `%26`, `=` became `%3D`, and `?` became `%3F`. It is the same idea as always, a public rule for packing characters into a restricted channel, using the hex byte values you already know.

URL encoding matters hugely in web security (course 11). Attackers use it to **sneak dangerous characters past filters**: a filter looking for a literal `<` might miss `%3C`, which the browser later turns back into `<`. The same bytes, read at two different stages with two different rules, is the ambiguity theme from topic 3, and it is a real attack technique.

8. Why Encoding Matters in Security

Encoding seems like a dry, technical topic, but it sits under a surprising amount of security:

- **Reading data in the wild.** Tokens, configuration, and captured traffic are full of Base64, hex, and URL-encoding. Recognizing and decoding them is a daily skill (and a whole category of CTF challenges).
- **Encoding is not protection.** Mistaking Base64 or hex for encryption is a classic error that leaves secrets exposed.
- **Ambiguity is an attack surface.** When two parts of a system decode the same bytes differently, attackers slip dangerous input through the gap, injection, filter bypass, and "smuggling" attacks all exploit this.
- **Correct text handling is a security control.** Knowing that one character is not one byte prevents length bugs, truncation bugs, and validation bypasses, especially with non-English text like Khmer.

You now understand how every kind of information, numbers (lesson 2) and text (this lesson), becomes bytes. Next, we leave the world of pure data and look at the machine that works on it: the CPU, memory, and storage.

Common Mistakes

1. Thinking Base64 (or hex) is encryption. It is not. It provides zero secrecy and is reversible by anyone instantly. Never "protect" a secret by encoding it.

2. Confusing the character `'0'` with the number 0. The character `'0'` is ASCII 48. Mixing them up causes real bugs.

3. Assuming one character is one byte. In UTF-8, non-ASCII characters (including every Khmer letter) are several bytes. Counting and chopping text by bytes breaks it.

4. Ignoring which encoding is in use. The same bytes read with the wrong table become garbage, or, worse, become a different character an attacker chose.

5. Trusting a filter that checks only one encoding. Attackers URL-encode or otherwise re-encode dangerous characters to slip past filters that look for the literal form.

Summary

Term	Remember
Character encoding	A public table mapping characters to numbers/bytes
ASCII	The original: basic English in 0–127; `H` = 72 = `0x48`
Unicode	One giant table; every character has a code point
UTF-8	The common rule to store code points as bytes; ASCII-compatible; 1–4 bytes per character
One char ≠ one byte	Non-ASCII (incl. Khmer) take 2–4 bytes; appearance, code points, and bytes differ
Base64	Represents binary as safe text; **encoding, not encryption**; reversible by anyone
URL/percent encoding	`%` + hex byte for special characters; used to bypass filters
Encoding vs encryption	Public rule vs secret key; encoding never protects

Check Your Understanding

1. Why is the letter `H` stored as the byte `0x48`?
2. What is the ASCII value of the digit character `'0'`, and why is that surprising?
3. What does UTF-8 do, and why is it compatible with ASCII?
4. In UTF-8, how many bytes does a typical Khmer letter take? What does that mean for counting characters?
5. A token in a config file is `TWluYSBDb2ZmZWU=`. Is the data secret? What would you do?
6. In a URL you see `%3C`. What character is that, and why might an attacker use it?
7. State the one-sentence difference between encoding and encryption.

Answers

1. Because the ASCII (and Unicode) standard table assigns the character `H` the number 72, which is `0x48` in hex, and everyone agreed to use that table.
2. 48. It's surprising because the character `'0'` is not the number 0; they are different things, and confusing them causes bugs.
3. UTF-8 turns Unicode code points into bytes. It's ASCII-compatible because every ASCII character (0–127) is stored as the same single byte as in ASCII, so existing English text is already valid UTF-8.
4. Typically three bytes. It means one character is not one byte, so counting, measuring, or chopping text by bytes can split or miscount Khmer characters.
5. No, it is not secret, it is Base64, which anyone can decode instantly (it decodes to "Mina Coffee"). You would decode it with `base64 -d` to read it, and never rely on Base64 to hide anything.
6. `%3C` is `<` (byte `0x3C`). An attacker uses it to slip a dangerous character past a filter that only looks for the literal `<`, since the browser decodes it back to `<` later.
7. Encoding is a public, reversible rule for representing data (no secrecy); encryption uses a secret key to make data unreadable without that key.

Practice

1. The word `Hi` is two ASCII characters. Using topic 2's values (`H` = 72 = `0x48`), find the byte for `i` (hint: it comes after `H` in... actually look it up or reason from `a` = 97). Then predict the hex dump of `Hi`.
2. Decide for each: is it **encoding** or **encryption**? (a) Base64; (b) a password-protected ZIP; (c) URL percent-encoding; (d) hex.

3. If you have a terminal: run `printf 'Khmer: ក' | xxd` and count how many bytes the Khmer letter takes versus the English letters.

A Worked Example

1. `i` is lowercase; from the demo, `i` is 105 (you can get it from `ord("i")`), which is `0x69`, matching the lesson-2 dump of `Hi!`. So `Hi` dumps as `48 69`, with `Hi` on the right.
2. (a) encoding; (b) encryption (a key/password makes it unreadable); (c) encoding; (d) encoding. Only the ZIP provides secrecy.
3. The English letters and the space are one byte each; the Khmer letter `ក` is three bytes (`e1 9e 80`), so the single Khmer character takes three times the space of an English one.

Course mark icon from Material Symbols by Google, Apache License 2.0.